King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 12
String Matching

Term 252

Reading Assignment

- Data Structures and Algorithms in Java, 4th Edition, Adam Drozdek, Cengage Learning, ISBN 978-9814239233
  - Chapter 13
    - Section 13.1.1 Straightforward Algorithm
    - Section 13.1.2 The Knuth-Morris-Pratt Algorithm

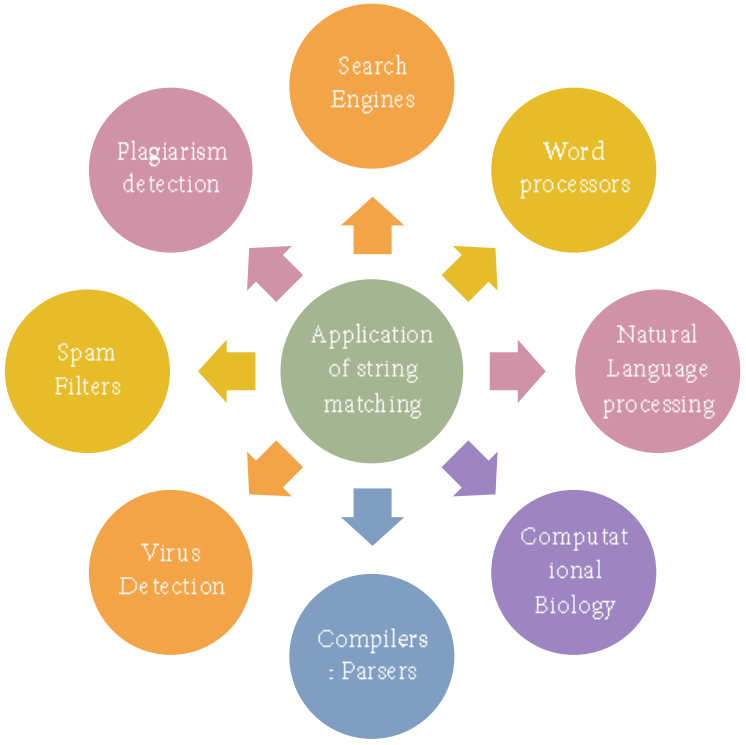
Outline

- Introduction
- Naive (Brute Force) Algorithm
- Knuth-Morris-Pratt (KMP) Algorithm
- Comparison of Brute Force and KMP Algorithms
- Some applications of String matching

One thing to take away from today's lecture: Even something as simple searching for a sub-string can be improved

Introduction

- The (Exact) String Matching Problem: Given two strings, a text T and a pattern P, find all occurrences of P in T.
- For example, the pattern "am" occurs two times (At positions 12 and 55) in the following string: "This is a sample sentence used in order to show the example"



- Notations:
  - T is the text,
  - P is the pattern to be searched in text T.
  - |T| and |P| : lengths of T and P respectively,
  - T_i is the character at index i,
  - P_j is the character at pattern index j

Brute Force Algorithm...

- Here is an example of the execution of the brute force algorithm:
  - Text: THIS IS A SIMPLE EXAMPLE
  - Pattern: SIMPLE
- Note: there is only one occurrence of P in the text T.

1. Start at index 0
2. Compare pattern with text characters one by one
3. If all match ⟶ found
4. If mismatch ⟶ shift pattern by 1 position
5. Repeat until end

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	COMPARISONS
T	H	I	S		I	S		A		S	I	M	P	L	E		E	X	A	M	P	L	E	
S	I	M	P	L	E																			
	S	I	M	P	L	E																		
		S	I	M	P	L	E																	
			S	I	M	P	L	E																
				S	I	M	P	L	E															
					S	I	M	P	L	E														
						S	I	M	P	L	E													
							S	I	M	P	L	E												
								S	I	M	P	L	E											
									S	I	M	P	L	E										
										S	I	M	P	L	E									

Brute Force Algorithm...

- Text: A B A C A B A A B C A B
- Pattern: A B A B

0	1	2	3	4	5	6	7	8	9	10	11	COMPARISONS
A	B	A	C	A	B	A	A	B	C	A	B	
A	B	A	B									

Naive (Brute Force) Algorithm

- By default, ⟶ overlapping; shift by 1 to the right
- Complexity Analysis
  - Outer loop: number of iterations is |T|- |P| +1
  - Inner loop: worst case |P|
  - Total:
    - worst case (|T|- |P| +1) *|P| ≈ O (|T||P|) , best case (|T|- |P| +1)

```
bruteForceStringMatching(pattern P, text T)
    i = 0;
    while i ≤ |T| - |P|
        j = 0;
        while T_i == P_j and j < |P|
            i++;          // try to match all characters in P;
            j++;
        if j == |P|
            return match at i - |P|; // success if the end of P is reached;
                          // if there is a mismatch,
        i = i - j + 1;    // shift P to the right by one position;
    return no match;      // failure if fewer characters left in T than |P|;
```

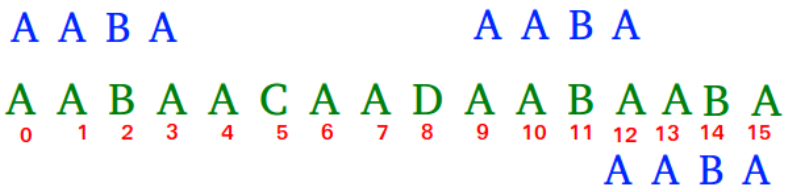
The Knuth-Morris-Pratt Algorithm

- Invented by Knuth, Morris and Pratt in 1977.
- Efficient for single pattern matching.
- The algorithm can be used for searching both overlapping and non-overlapping patterns in a text. For example,

Text : A A B A A C A A D A A B A A B A

Pattern : A A B A

  - Using overlapped pattern search, the pattern is found at Text indexes 0, 9, and 12
  - Using non-overlapped pattern search, the pattern is found at Text indexes 0 and 9 or 0 and 12.



- Several variants are available. We use the textbook model.
- It has two phases:
  - preprocessing the pattern and
  - searching the text.

- It is based on the observation that searching for a pattern in a text can be done by shifting the search by multiple positions based on pattern analysis.

KMP Algorithm - Motivation

- Consider the problem of finding instances of the string “abababa" in the text "ababcdabbabababab" using the brute-force algorithm.

Text	a	b	a	b	c	d	a	b	b	a	b	a	b	a	b	a	b
Pattern	a	b	a	b	a	b	a										
Step 1	✓																
Step 2	✓	✓															
Step 3	✓	✓	✓														
Step 4	✓	✓	✓	✓													
	✓	✓	✓	✓	✘												

Since ‘c’ does not match with `a' the search for pattern shifts by one position in the brute-force algorithm

- However, consider the pattern until the unmatched character, viz.,

  a b a b a

  - Note that the pattern has same prefix and suffix
  - This means that there is no need to match characters of the prefix of the pattern with its suffix if we already know that they are equal.

    a b a b c d a b b a b a b a b a b i = 4

    a b a b a b a b a j = 2

KMP Algorithm - Mechanism

- The KMP (Knuth-Morris-Pratt) Algorithm
  - eliminates this inefficiency by analyzing prefixes and suffixes within the pattern.
  - uses knowledge of these patterns to restart the search by eliminating redundant searches.

- Note: Unlike the Brute force algorithm, the KMP algorithm does not reset the index i of the text T.

Main idea is to skip some comparisons by using the previous comparison result

KMP Algorithm – Preprocessing Terms

- A **proper prefix** of a string **P[0..n−1]** is any substring that starts at index 0 and ends at any position **before the last character;** a substring P[0..j] where j varies from 0 to n-2
  - Example: for "ESTATES", proper prefixes are:

    λ E ES EST ESTA ESTAT ESTATE
- A proper suffix of a string P[0..n−1] is any substring that ends at the last character and starts at any position after the first character. a substring P[$j..n-1$] where j varies from 1 to n-1
  - Example: for "ESTATES", proper suffixes are:

    λ S ES TES ATES TATES STATES
- λ denoted the empty string
- A **border** is the longest substring that is **both** a proper prefix **and** a proper suffix.
  Example: For the string " ESTATES ", the border is ES
- Find the border:
  - BABABABA

KMP Algorithm - Preprocessing

- We construct the **next-array** (lps array, π –array, prefix table, or failure function) on the pattern to apply the KMP algorithm.

- The element at index **j** of the **next array** indicates the length of the border (or the length of the lps) of the pattern P[0..j – 1] (next[0] is defined as –1).

- More formally, a **next-array** with sentinel

$$next[j] = \begin{cases} -1 & \text{for } j = 0 \\ \max\{k{:}0 < k < j \text{ and } P[0\ldots k-1] = P[j-k\ldots j-1]\} & \text{if such a } k \text{ exists} \\ 0 & \text{otherwise} \end{cases}$$

If the border is λ

KMP Algorithm – The lps- or next-array

{-1, 0, 0, 0, 1, 1, 2}

- For the pattern P = "ABCAABC", the next array can be found as:

j	P[0..j-1]	Proper Prefixes		Proper Suffixes	next[j]
0					
1					
2					
3					
4					
5					
6					

j	P[0..j-1]	Proper Prefixes		Proper Suffixes	next[j]
0	-	-		-	-1
1	A	λ		λ	0
2	AB	A		B	0
3	ABC	A, AB		C,BC	0
4	ABCA	A, AB, ABC		A,CA,BCA	1
5	ABCAA	A, AB, ABC, ABCA		A,AA,CAA,BCAA	1
6	ABCAAB	A, AB, ABC, ABCA, ABCAA		B, AB, AAB, CAAB, BCAAB	2

KMP Algorithm – The lps- or next-array

-1 0 0 1 2 3 4 0 1 1

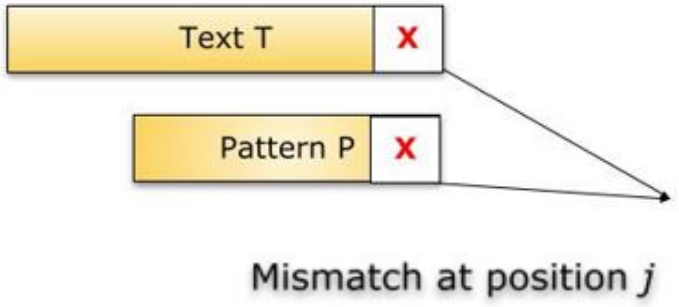
- For the pattern P = "BABABACBBA", the next array can be found as:

j	P[0..j-1]	Proper Prefixes		Proper Suffixes	next[j]
0					
1					
2					
3					
4					
5					
6					
7					
8					
9					

j	P[0..j-1]	Proper Prefixes		Proper Suffixes	next[j]
0	-	-		-	-1
1	B	λ		λ	0
2	BA	B		A	0
3	BAB	B, BA		B, AB	1
4	BABA	B, BA, BAB		A, BA, ABA	2
5	BABAB	B, BA, BAB, BABA		B, AB, BAB, ABAB	3
6	BABABA	B, BA, BAB, BABA, BABAB		A, BA, ABA, BABA, ABABA	4
7	BABABAC	B, BA, BAB, BABA, BABAB, BABABA		C, AC, BAC,ABAC, BABAC, ABABAC	0
8	BABABACB	B, BA, BAB, BABA, BABAB, BABABA, BABABAC		B, CB, ACB,BACB, ABACB,BABACB, ABABACB,	1
9	BABABACBB	B, BA, BAB, BABA, BABAB, BABABA, BABABAC, BABABACB		B,BB, CBB, ACBB, BACBB, ABACBB, BABACBB, ABABACBB	1

The KMP Algorithm – Principle Explained

- Consider the following configuration where there is a character mismatch at index i of the text and index j of the pattern.
- The shaded part of the pattern matches with the text.



- Suppose that the matching part of the pattern i.e.

P[0..j – 1] has a border that is

both the longest proper prefix

and the longest proper suffix.

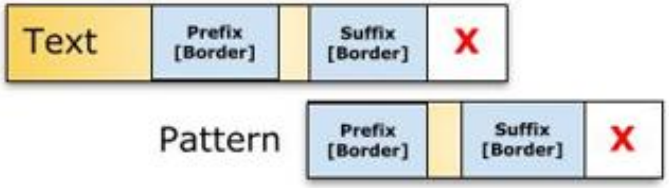


- The search for the pattern can be resumed by aligning the
- longest proper prefix of P[0..j – 1]

with the

longest proper suffix of the matching

part of T.



- This realignment eliminates redundant searches.

KMP Algorithm Search Methodology

```
Let T be the text
Let P be the pattern.
If there is a mismatch at index j > 0 of P
    P is shifted j – next[j] to the right of T;
    Restart searching P at index next[j];
Else // there is a mismatch at index j == 0
    P is shifted 1 to the right of text T; // same as j – next[j] == 0 – (-1)
    Restart searching the pattern at index 0.
```

Match:

i++; j++;

Mismatch:

Shift the pattern to the right by j – next[j] characters;

Resume pattern search at max{0, next[j]} pattern index.

Index j	0	1	2	3	4	5	6	7	8	9	10
Pattern[j]	A	A	B	A	A	C	A	A	B	A	C
next[j]	-1	0	1	0	1	2	0	1	2	3	4

The KMP Algorithm – Tracing example 01…

Index j	0	1	2	3	4
Pattern[j]	A	B	A	B	X
next[j]	-1	0	0	1	2

Shift the pattern to the right by j – next[j] characters;
Resume pattern search at max{0, next[j]} pattern index.

Step	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	comparisons
	A	B	T	A	B	A	B	Y	A	B	A	B	X	K	A	B	A	B	X	Z	M	
1																						
2																						
3																						
4																						
5																						
6																						

- Example: Trace the KMP algorithm in searching for the pattern ABABX in the string:
- ABTABABYABABXKABABXZM by finding the first occurrence of the pattern, if it exists.

The KMP Algorithm – Tracing example 01…

Index j	0	1	2	3	4
Pattern[j]	A	B	A	B	X
next[j]	-1	0	0	1	2

Shift the pattern to the right by j – next[j] characters;
Resume pattern search at max{0, next[j]} pattern index.

Step	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	comparisons
	A	B	T	A	B	A	B	Y	A	B	A	B	X	K	A	B	A	B	X	Z	M	
1	A	B	A	B	X																	3
2			A	B	A	B	X															1
3				A	B	A	B	X														5
4						A	B	A	B	X												1
5								A	B	A	B	X										1
6									A	B	A	B	X									5

- I = 0 and j = 0; IF T[i] == P[j]: i++; j++;
- Step 1: j == 2, In step 2, shift pattern by j – next[j] = 2 – 0 = 2, resume pattern search at pattern index → J = max(0 , next[j])= max(0 ,0)= 0
- Step 2: j == 0, In step 3, shift pattern by 0 – (-1) = 1, resume pattern search at pattern index → J = max(0 , next[j])= max(0 ,-1)= 0
- Step 3: j == 4, In step 4, shift pattern by 4 – 2 = 2, resume pattern search at pattern index → J = max(0 , next[j])= max(0 ,2)= 2
- Step 4: j == 2, In step 5, shift pattern by 2 – 0 = 2, resume pattern search at pattern index → J = max(0 , next[j])= max(0 ,0)= 0
- Step 5: j == 0, In step 6, shift pattern by 0 – (-1) == 1, resume pattern search at pattern index → J = max(0 , next[j])= max(0 ,-1)= 0
- Step 6: **j == 4**, First occurrence of Pattern found.
- If the search is for non-overlapping patterns, shift pattern by length(pattern) == 5, Else if the search is for overlapping patterns, shift the pattern by 1
- Resume the pattern search at pattern index. 0

The KMP Algorithm – Tracing example 02

- Given the following search pattern: AABAAC
- Create the KMP next or lps array. Using the search pattern and the next array you created, perform a search on the following block of text: AACAABAABAACAACB

j	0	1	2	3	4	5
Pattern	A	A	B	A	A	C
next[j]	-1	0	1	0	1	2

Shift the pattern to the right by j – next[j] characters;
Resume pattern search at max{0, next[j]} pattern index.

0	1	2	3	4	5	6	7	8								#comparisons	j, next[j]	pattern shift	pattern index
A	A	C	A	A	B	A	A	B	A	A	C	A	A	C	B				
A	A	B	A	A	C											3	<2,1>	1	1
	A	A	B	A	A	C										1	<1,0>	1	0
		A	A	B	A	A	C									1	<0,-1>	1	0
			A	A	B	A	A	C								6	<5,2>	3	2
						A	A	B	A	A	C					4	<5,2>		
																	return 6		

The KMP Algorithm

```
kmpSearch(T,P) {
        findNext(P,next);
        int i = 0;  // text index
        int j = 0;  // pattern index

        while (i < |T| && j < |P|) {
            if (j == -1 || T[i] == P[j]) {
                i++; j++;
            } else
                j = next[j];
        }
        if (j == |P|) {
            return i - j;  // starting index of match
        } else {
            return -1;     // no match
        }
    }
```

Index j		0	1	2	3	4	5	6	7	8	9	10
Pattern[j]		A	A	B	A	A	C	A	A	B	A	C
next[j]		-1	0	1	0	1	2	0	1	2	3	4

```
findNext( P, next) {
        next[0] = -1;
        int i = 0;
        int j = -1;

        while (i < |P| - 1) {
            if (j == -1 || P[i] == P[j]) {
                i++;    j++;     next[i] = j;
            } else
                j = next[j];
        }
    }
```

$O(|P|)$

Algorithm	Preprocessing	Complexity	Multiple Pattern Search
Brute Force	-	$\Theta(\|P\|.\|T\|)$ in the worst case	No
Knuth Morris Pratt	$\Theta(\|P\|)$	$\Theta(\|T\|)$	No

Exercises

- Design an implementation of the Knuth Morris Pratt Algorithm by using the next-array for string matching.
  - Test your program by searching for the string "bacbaaa" in the text "bacbacabcbacbbbacabacbabcbbba". -1 0 0 0 1 2 0
- Given the following search pattern: AACAAABA -1 0 1 0 1 2 2 0
  - Create the KMP next array. Using the search pattern and the array you created, perform a search on the following block of text: AACAACAAABAY
- Given the following search pattern: ABXABYABXZ -1 0 0 0 1 2 0 1 2 3
  - Create the KMP next array. Using that search pattern and the array you created, perform a search on the following block of text: ABXABYABXABYABXZABC

Exercises...

- Create the next array for each of the following patterns:

"AAAA"	→ -1 0 1 2
"ABCDE"	→ -1 0 0 0 0
"AABAACAABAA"	→ -1 0 0 0 0 1 2 0 1 2
"AAACAAAAAC"	→ -1 0 1 2 0 1 2 3 3 3
"AAABAAA"	→ -1 0 1 2 0 1 2
"ABCXXXABCY"	→ -1 0 0 0 0 0 0 1 2 3
"ABABABA"	→ - 1 0 0 1 2 3 4
"ABCDABEABF"	→ -1 0 0 0 0 1 2 0 1 2
"ABCDEABFABC"	→ -1 0 0 0 0 0 1 2 0 1 2
"AABCADAABE"	→ -1 0 1 0 0 1 0 1 2 3
"AAAABAACD"	→ -1 0 1 2 3 0 1 2 0
"ABABD"	→ -1 0 0 1 2
"ABCAABC"	→ -1 0 0 0 1 1 2
"ABABX"	→ -1 0 0 1 2
"ACACAGT"	→ -1 0 0 1 2 3 0
"BABABACBBA"	→ -1 0 0 1 2 3 4 0 1 1